Software Requirements Specification (SRS)

- **Project: Hospital System**
- **Prepared by: Mostafa Mohammed el Sayed Mandour**

---

1. Introduction

1.1 Purpose

The objective of this document is to provide a comprehensive and precise description of all functional and non-functional requirements for the Hospital System. This system is designed to streamline interactions between patients and doctors while organizing financial and administrative operations.

1.2 Scope of the System

The system covers six primary modules:

1. **Identity Module:** Manages registration, login, and verification (Auth Service).
2. **Profile Module:** Manages demographic and professional data for patients and doctors.
3. **Appointment Module:** Handles booking, rescheduling, and cancellation of appointments.
4. **Billing Module:** Calculates fees and processes electronic payments.
5. **Medical Directory Module:** Facilitates searching for and categorizing doctors.
6. **Monitoring Module (AOP):** Tracks operations and records security logs.

---

2. Functional Modeling (Use Case Diagrams Analysis)

2.1 Authentication Service (Module 1)

This module allows users (Patients/Doctors) to securely access the system. Key use cases include:

- **Sign Up:** Create a new account and select a role.
- **Login:** Access the system using email and password.
- **Forgot Password:** Recover access via secure email tokens.

- **Admin Management:** Grants administrators the authority to modify or delete user accounts.

2.2 Profile Management (Module 2)

Focuses on demographic and professional information:

- **Update Profile:** Allows patients to update contact details and doctors to update specialties.
- **View Profile:** Enables the review of basic medical records and history.

2.3 Appointment Service (Module 3)

The core of the system, providing:

- **Book Appointment:** Selection of doctor, time, and date.
- **Reschedule/Cancel:** Modifying existing time slots.
- **Check Availability:** An automated system that ensures no scheduling conflicts occur.

2.4 Billing & Payment Service (Module 4)

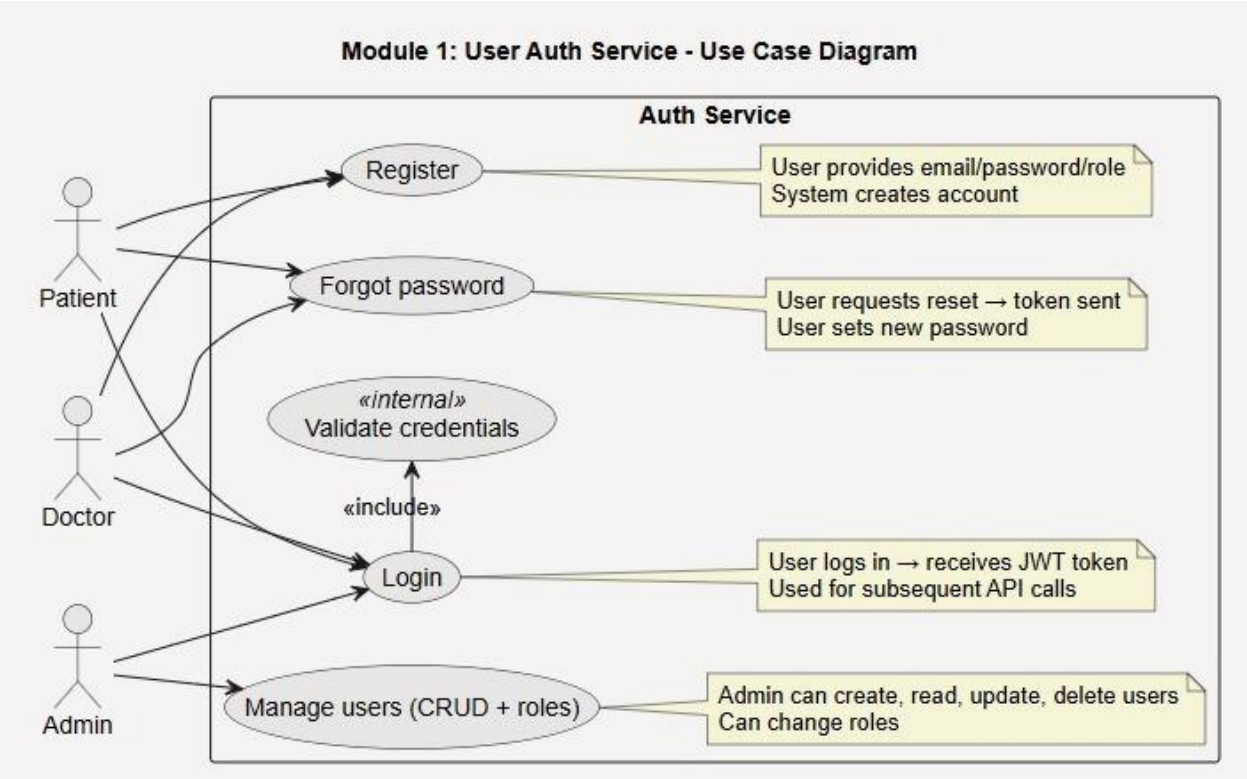
- **Generate Invoice:** Creating a bill after each medical consultation.
- **Make Payment:** Processing payments via credit cards or cash.
- **View Payment History:** A complete log of all financial transactions.

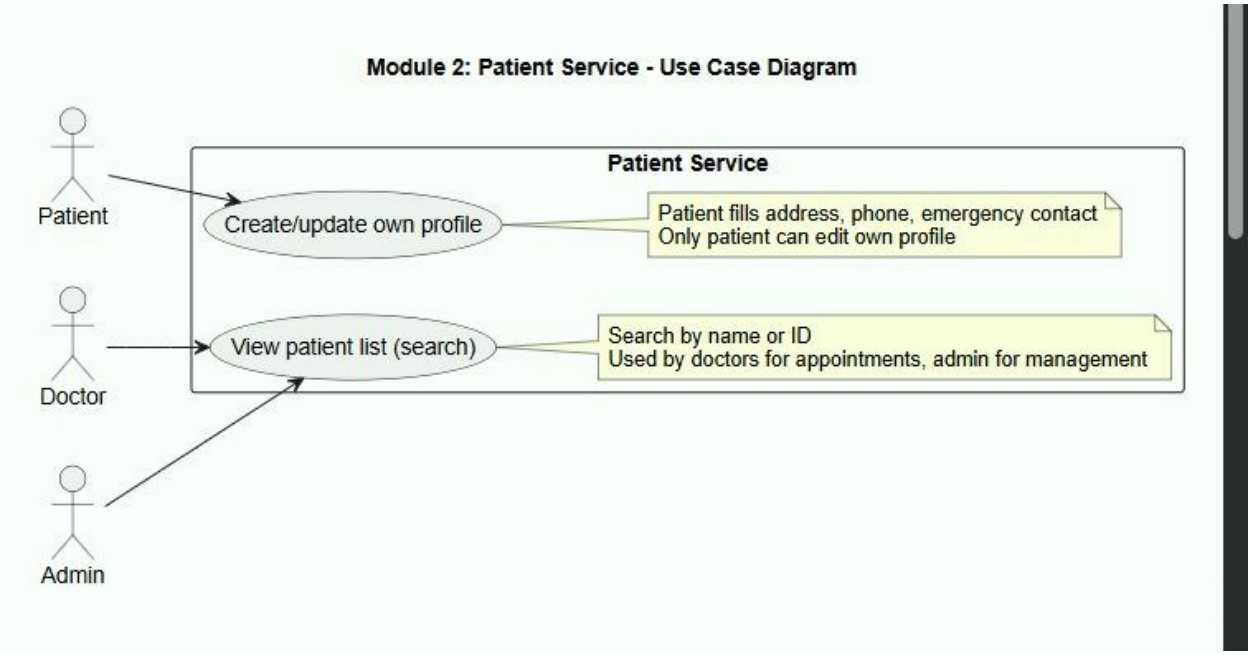
2.5 Monitoring & AOP (Module 6)

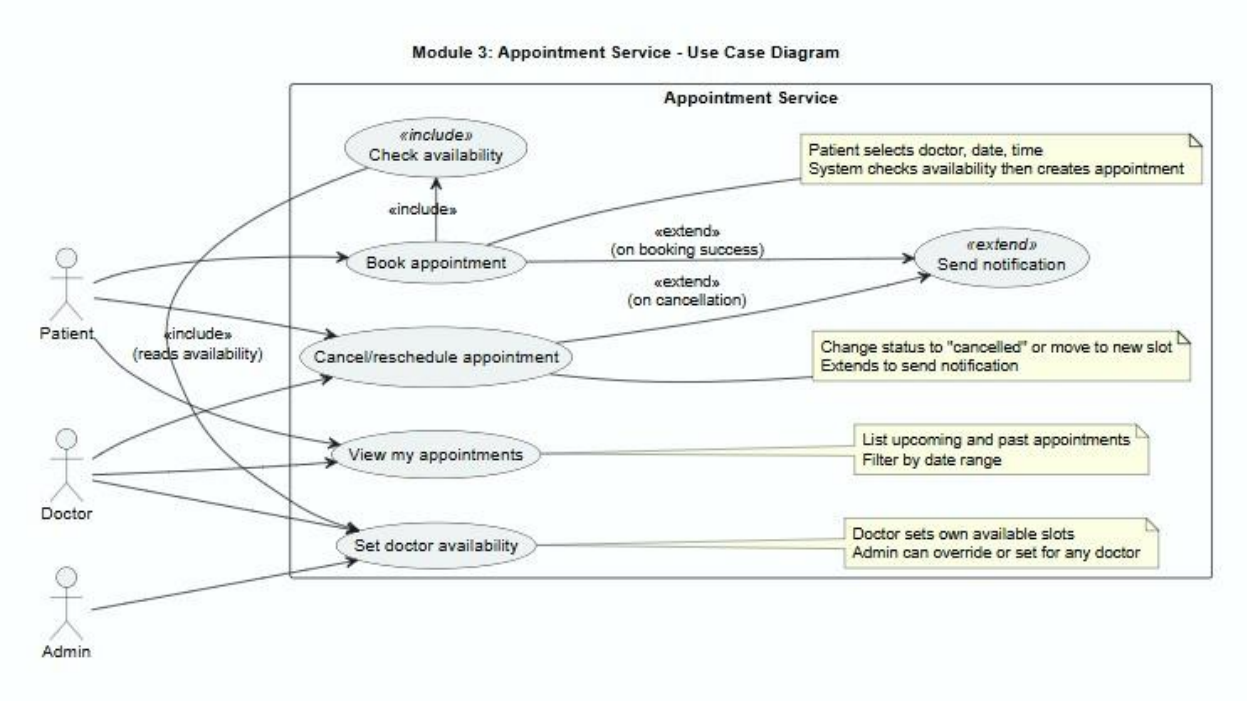
An advanced technical feature ensuring security:

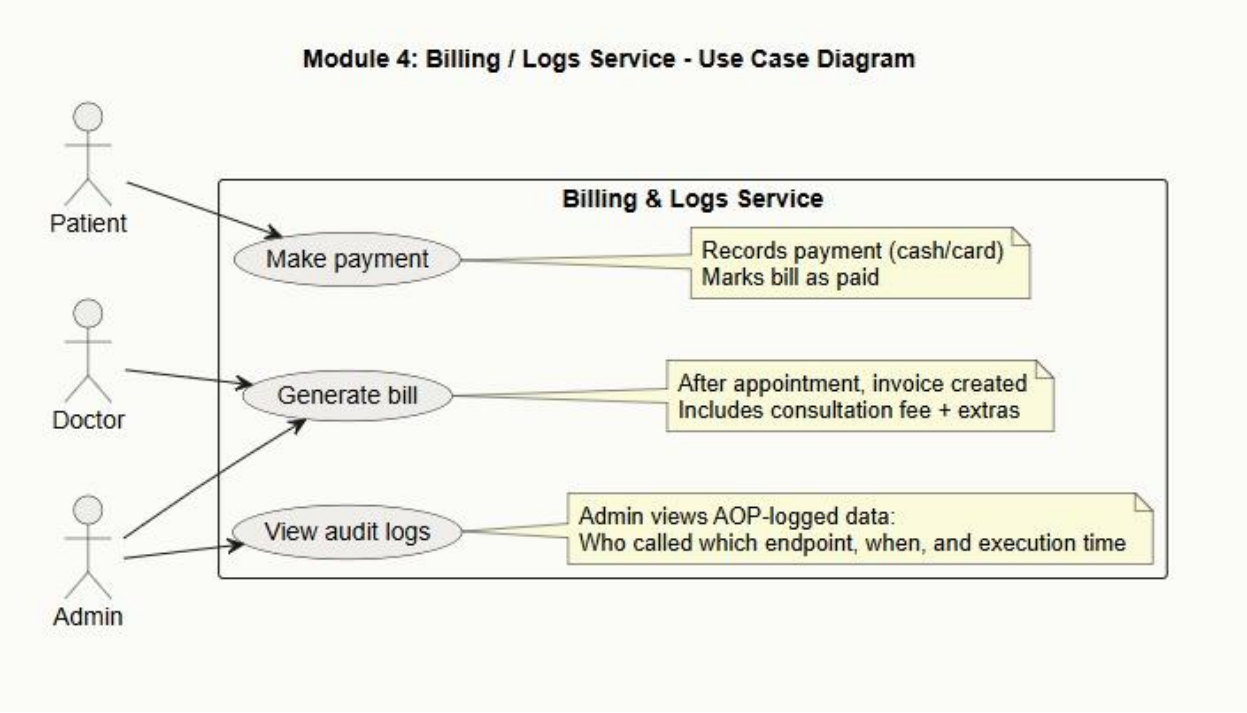
- **Logging:** Recording every login attempt and API request.
- **Performance Monitoring:** Measuring system response time for every operation.

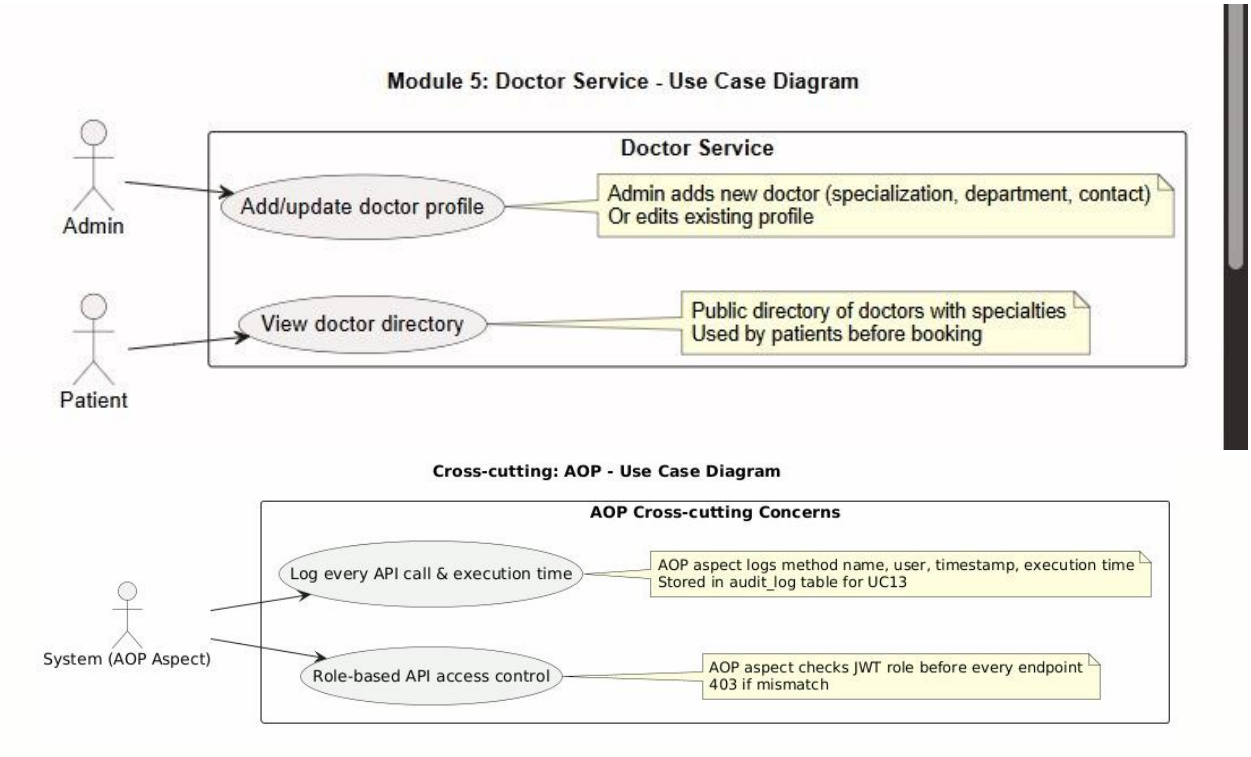
Use Case Diagram :


Module 1: User Auth Service - Use Case Diagram
Auth Service
Patient
Doctor
Admin
Register
Forgot password
«internal»
Validate credentials
«include»
Login
Manage users (CRUD + roles)
User provides email/password/role
System creates account
User requests reset → token sent
User sets new password
User logs in → receives JWT token
Used for subsequent API calls
Admin can create, read, update, delete users
Can change roles


Module 2: Patient Service - Use Case Diagram
Patient Service
Patient
Doctor
Admin
Create/update own profile
View patient list (search)
Patient fills address, phone, emergency contact
Only patient can edit own profile
Search by name or ID
Used by doctors for appointments, admin for management


Module 3: Appointment Service - Use Case Diagram
Appointment Service
«include»
Check availability
«include»
Book appointment
«extend»
(on booking success)
«extend»
(on cancellation)
«extend»
Send notification
Cancel/reschedule appointment
View my appointments
Set doctor availability
Patient selects doctor, date, time
System checks availability then creates appointment
Change status to "cancelled" or move to new slot
Extends to send notification
List upcoming and past appointments
Filter by date range
Doctor sets own available slots
Admin can override or set for any doctor
Patient
Doctor
Admin
«include»
(reads availability)


Module 4: Billing / Logs Service - Use Case Diagram
Billing & Logs Service
Make payment
Generate bill
View audit logs
Records payment (cash/card)
Marks bill as paid
After appointment, invoice created
Includes consultation fee + extras
Admin views AOP-logged data:
Who called which endpoint, when, and execution time
Patient
Doctor
Admin



---

3. Workflow Analysis (Activity Diagrams)

3.1 Login and Registration Flow

The activity begins at the authentication page where the user chooses between "Sign In" or "Sign Up". For registration, the system validates the email address. For login, it verifies credentials and issues a session "Token".

3.2 Booking Flow

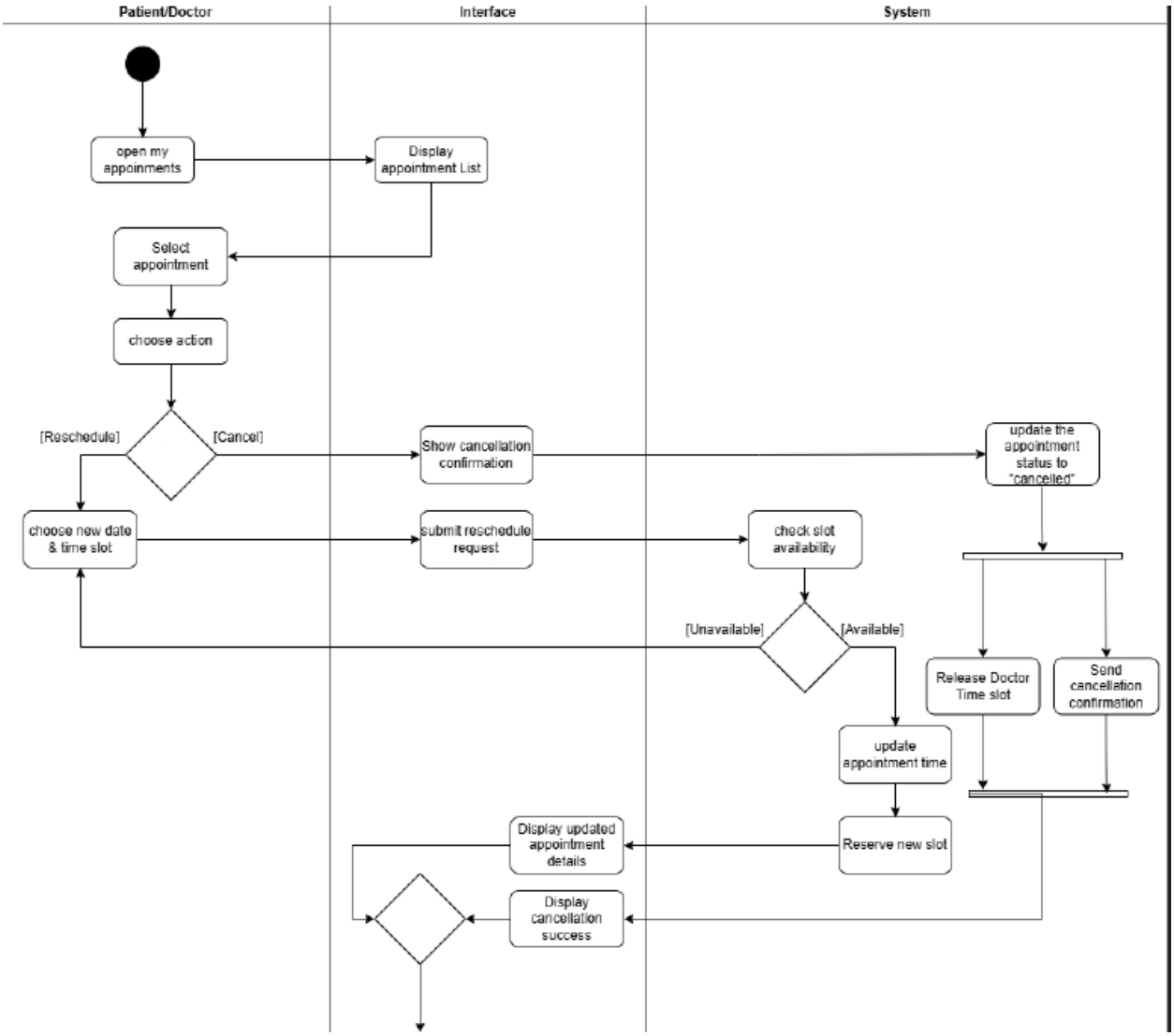
The process starts with a doctor search, followed by selecting a "Slot". The system hits a **Decision Point**:

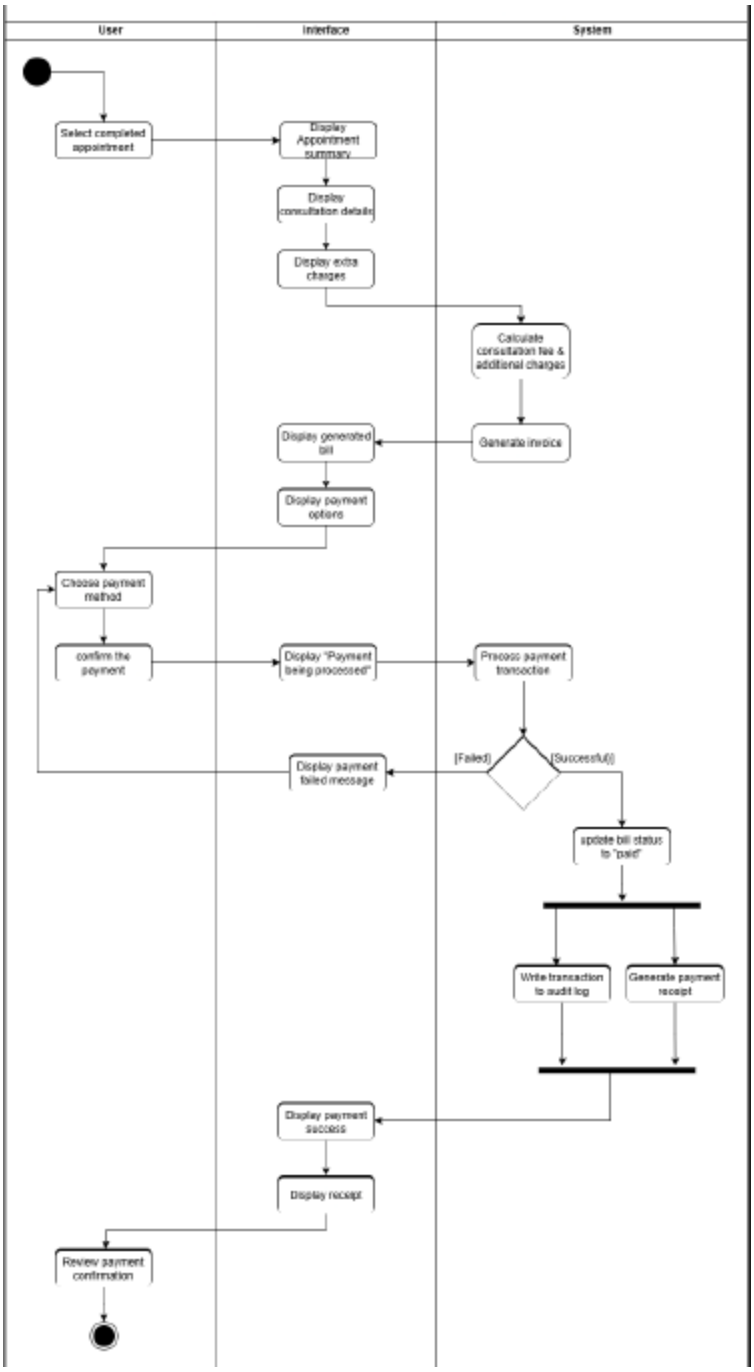
- If the time is **Available**: The appointment is created, and the doctor’s schedule is updated.
- If **Unavailable**: An alert is shown to the patient to choose another time.

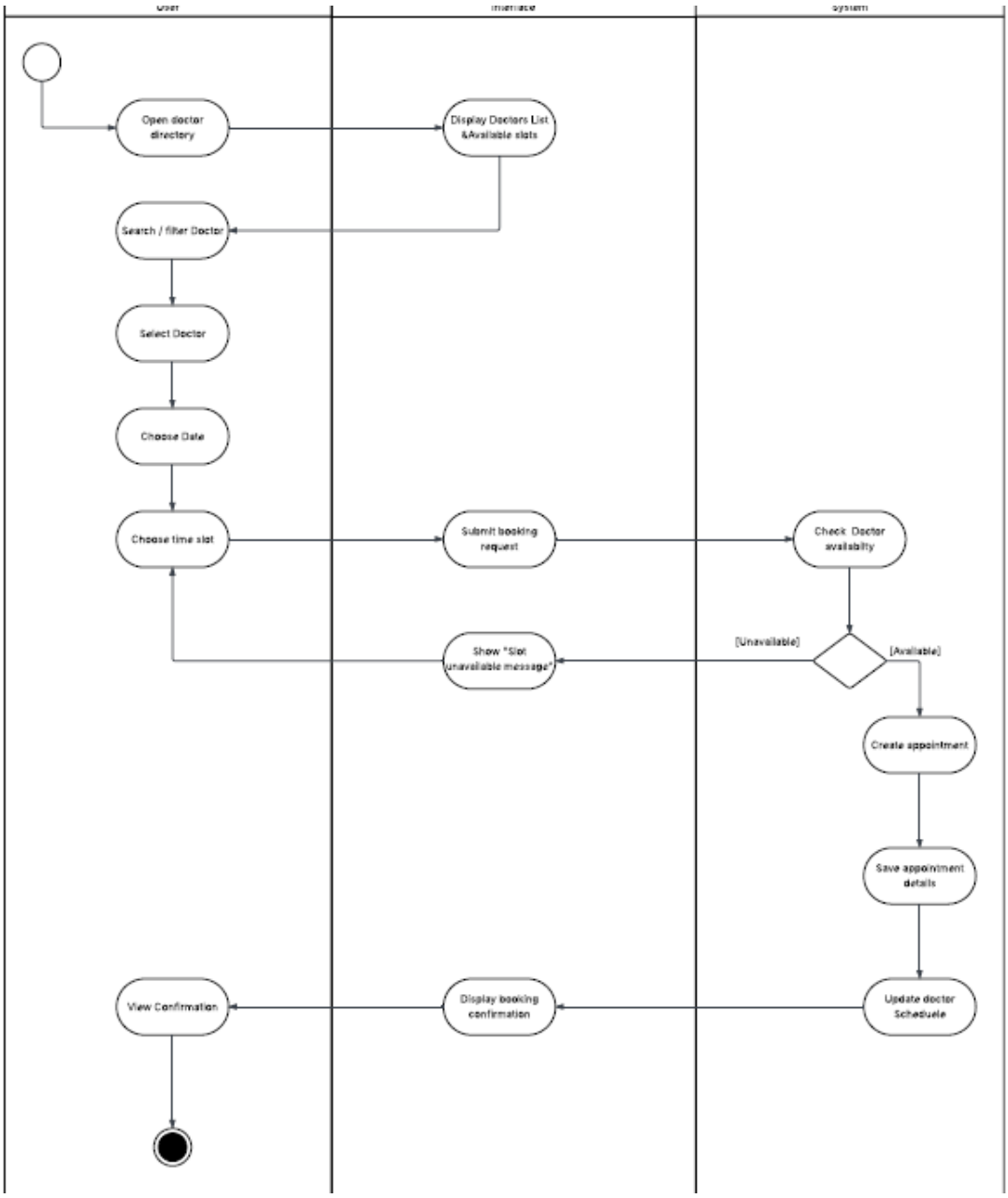
3.3 Billing & Payment Flow

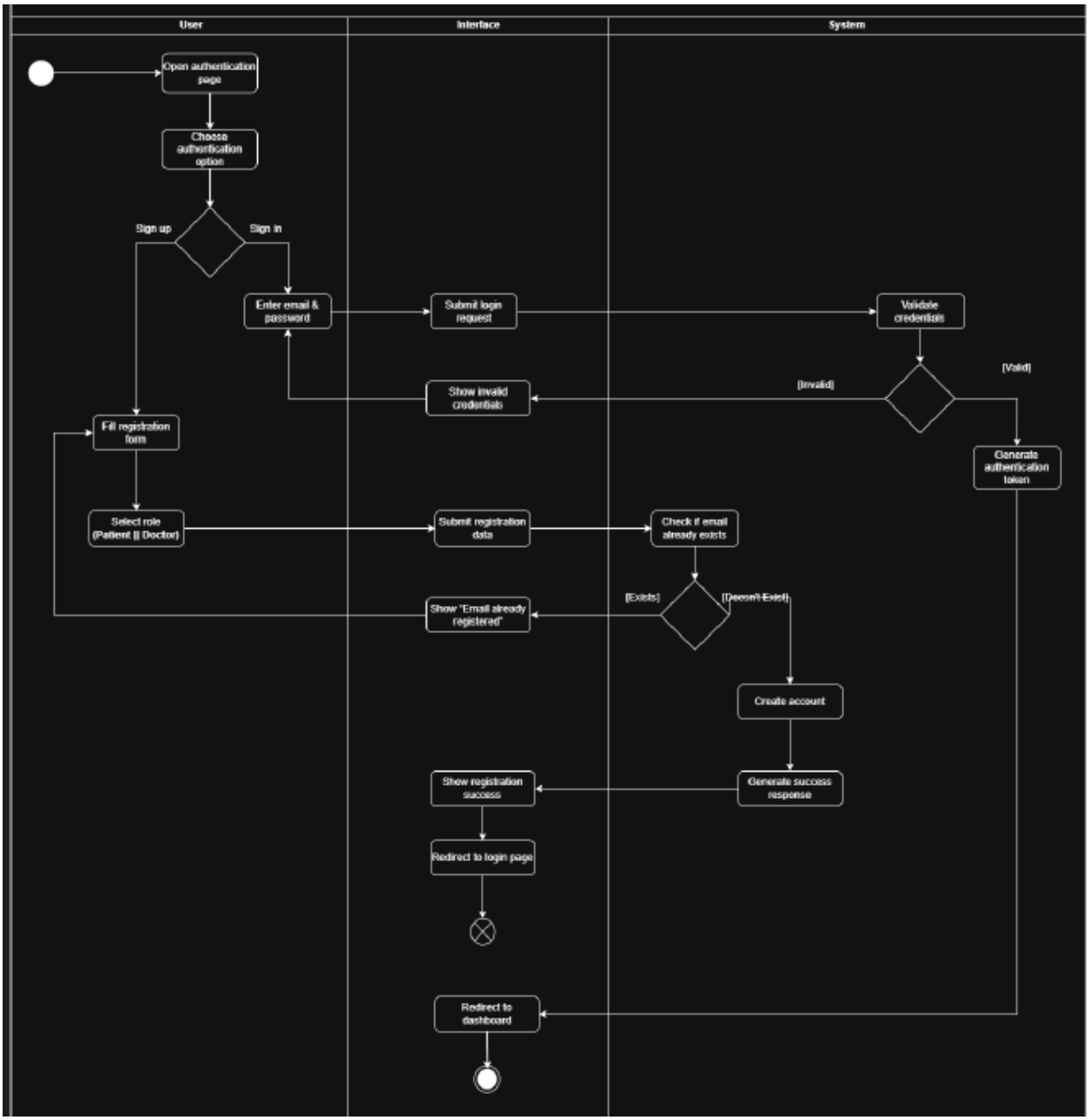
Starts after the consultation ends. The system calculates the base fee plus any additional charges. The user selects a payment method. If the payment fails, the system reverts to the payment method selection; if successful, the bill status is updated to "Paid" and recorded in the Audit Log.

Activity Diagram :




User
Interface
System
Select completed appointment
Display Appointment summary
Display consultation details
Display extra charges
Calculate consultation fee & additional charges
Generate invoice
Display generated bill
Display payment options
Choose payment method
confirm the payment
Display "Payment being processed"
Process payment transaction
[Failed]
[Successful]
Display payment failed message
update bill status to "paid"
Write transaction to audit log
Generate payment receipt
Display payment success
Display receipt
Review payment confirmation


User
Interface
System
Open doctor directory
Display Doctors List &Available slots
Search / filter Doctor
Select Doctor
Choose Date
Choose time slot
Submit booking request
Check Doctor availability
[Unavailable]
[Available]
Show "Slot unavailable message"
Create appointment
Save appointment details
View Confirmation
Display booking confirmation
Update doctor Schedule



---

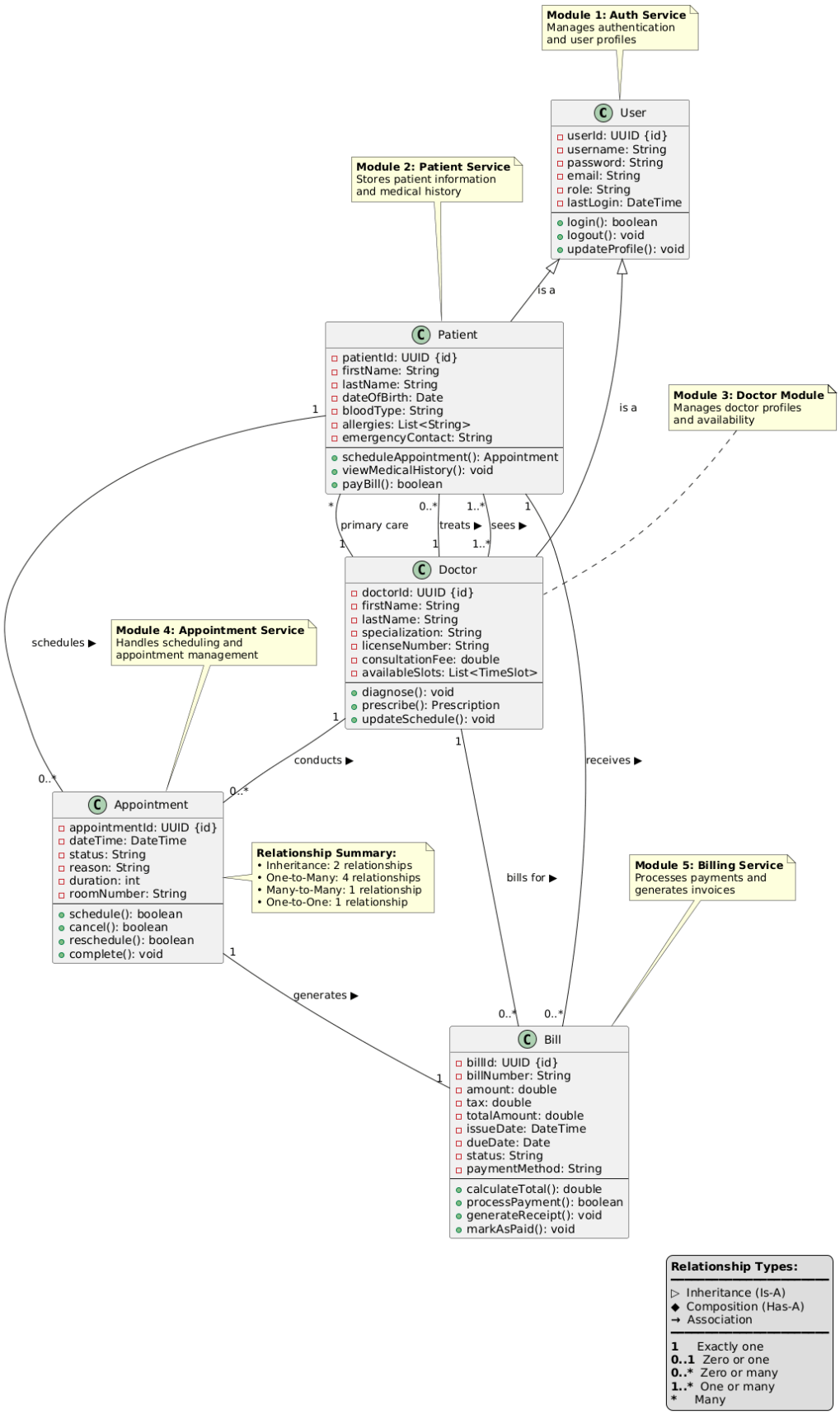
4. Structural Modeling (Class Diagram)

The system consists of the following primary entities:

1. **User:** Base class containing (ID, Name, Email).
2. **Patient & Doctor:** Classes inheriting from User with unique attributes (Specialty for doctors, Medical History for patients).

3. **Appointment:** Linked via a (1-to-Many) relationship with both Doctor and Patient.
4. **Invoice:** Associated with an appointment to define costs.
5. **Logging Service:** An independent class that records operations and timestamps.

Class diagram :


Module 1: Auth Service
Manages authentication and user profiles
User
userId: UUID {id}
username: String
password: String
email: String
role: String
lastLogin: DateTime
login(): boolean
logout(): void
updateProfile(): void
Module 2: Patient Service
Stores patient information and medical history
is a
Patient
patientId: UUID {id}
firstName: String
lastName: String
dateOfBirth: Date
bloodType: String
allergies: List<String>
emergencyContact: String
scheduleAppointment(): Appointment
viewMedicalHistory(): void
payBill(): boolean
is a
Module 3: Doctor Module
Manages doctor profiles and availability
1
*
primary care
1
0..*
treats ▶
1
1..*
sees ▶
1..*
1
Doctor
doctorId: UUID {id}
firstName: String
lastName: String
specialization: String
licenseNumber: String
consultationFee: double
availableSlots: List<TimeSlot>
diagnose(): void
prescribe(): Prescription
updateSchedule(): void
schedules ▶
Module 4: Appointment Service
Handles scheduling and appointment management
1
conducts ▶
0..*
0..*
1
receives ▶
Appointment
appointmentId: UUID {id}
dateTime: DateTime
status: String
reason: String
duration: int
roomNumber: String
schedule(): boolean
cancel(): boolean
reschedule(): boolean
complete(): void
Relationship Summary:
• Inheritance: 2 relationships
• One-to-Many: 4 relationships
• Many-to-Many: 1 relationship
• One-to-One: 1 relationship
bills for ▶
Module 5: Billing Service
Processes payments and generates invoices
1
generates ▶
0..*
0..*
1
Bill
billId: UUID {id}
billNumber: String
amount: double
tax: double
totalAmount: double
issueDate: DateTime
dueDate: Date
status: String
paymentMethod: String
calculateTotal(): double
processPayment(): boolean
generateReceipt(): void
markAsPaid(): void
Relationship Types:
▷ Inheritance (Is-A)
◆ Composition (Has-A)
→ Association
1 Exactly one
0..1 Zero or one
0..* Zero or many
1..* One or many
* Many

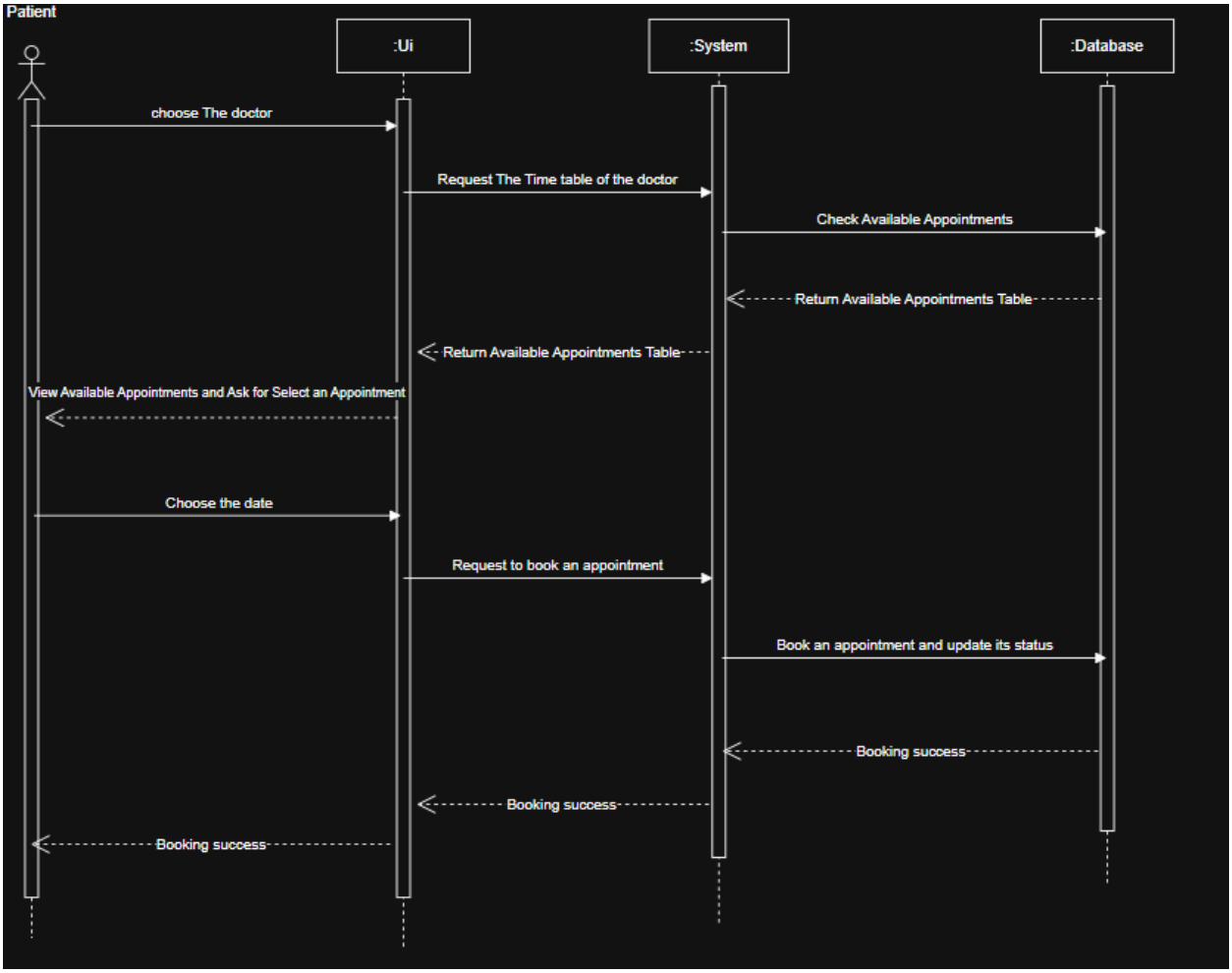
---

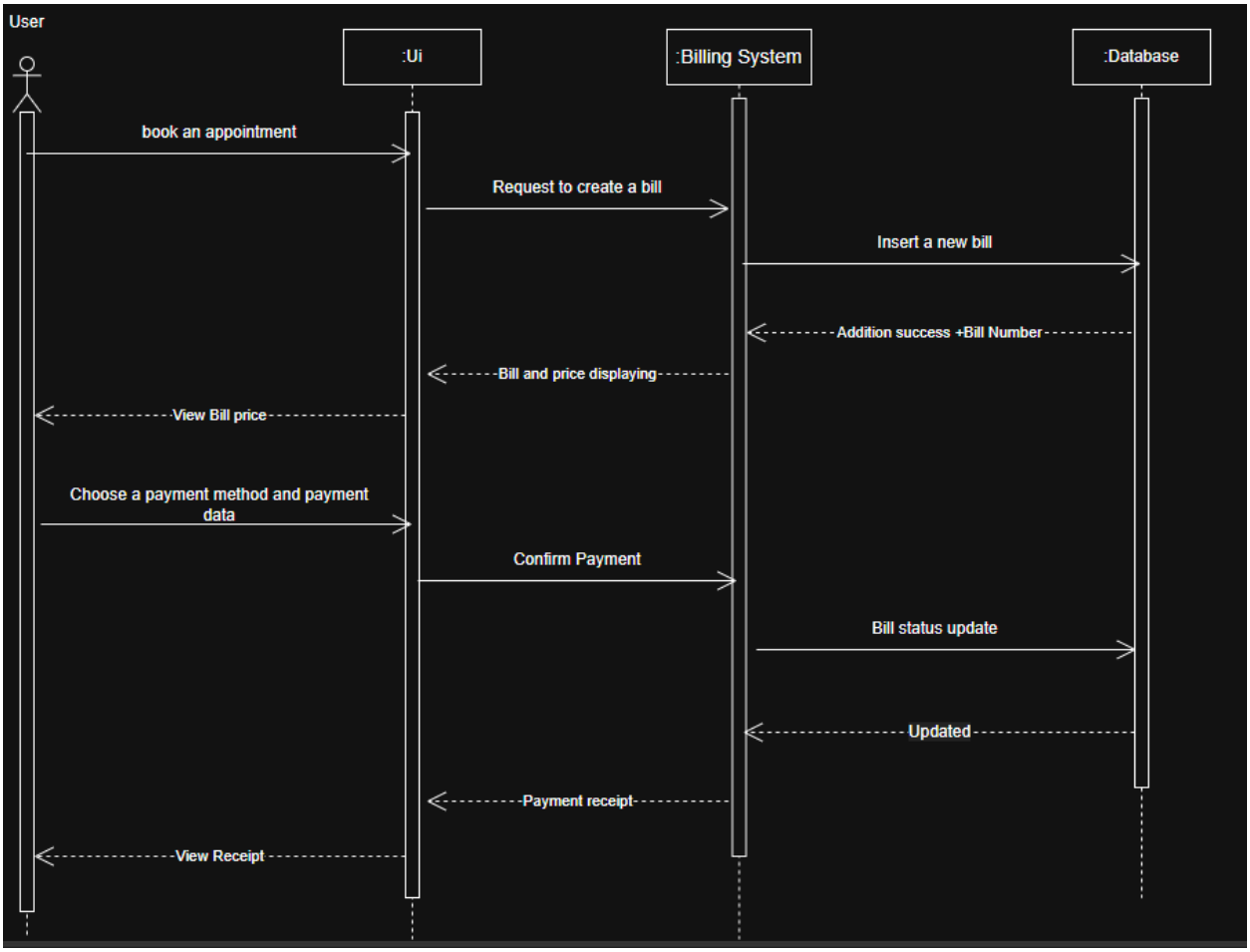
5. Behavioral Modeling (Sequence Diagrams)

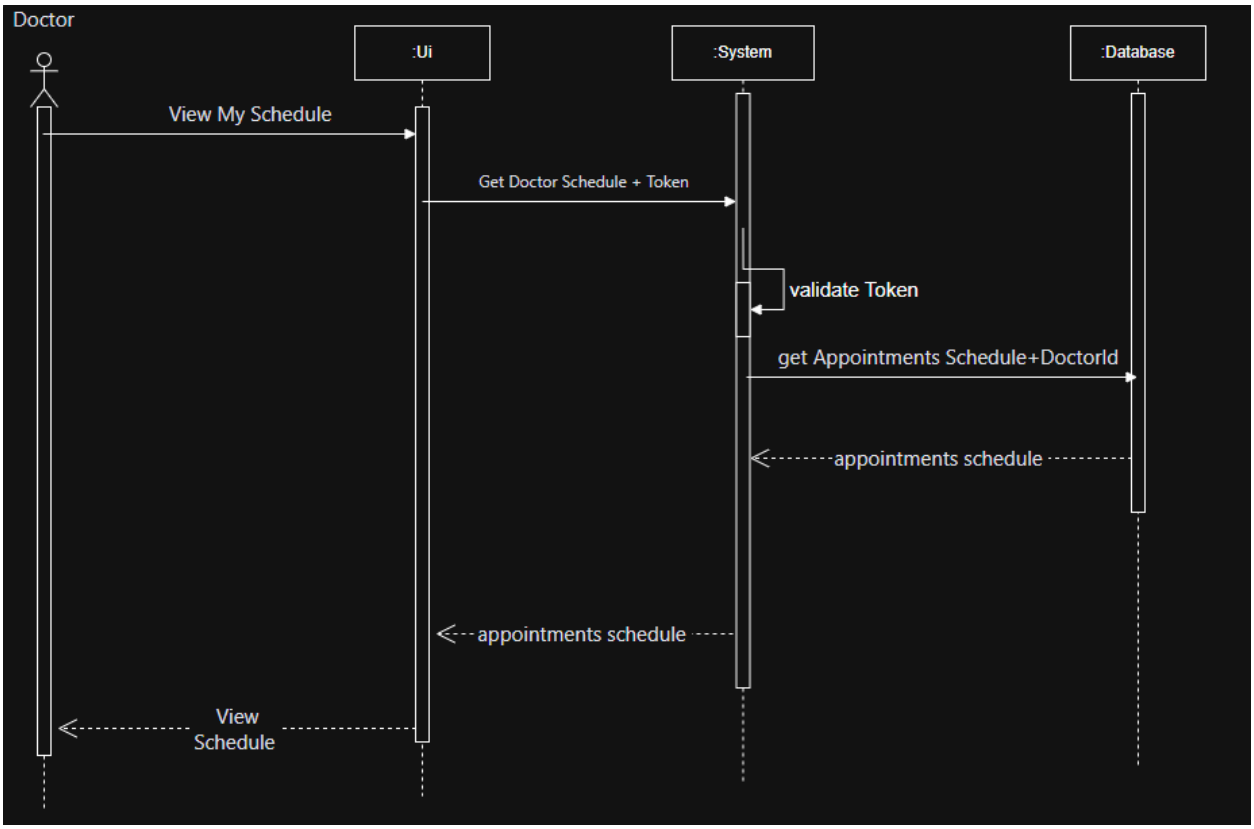
These diagrams illustrate the communication between the User Interface, Service Layer, and Database:

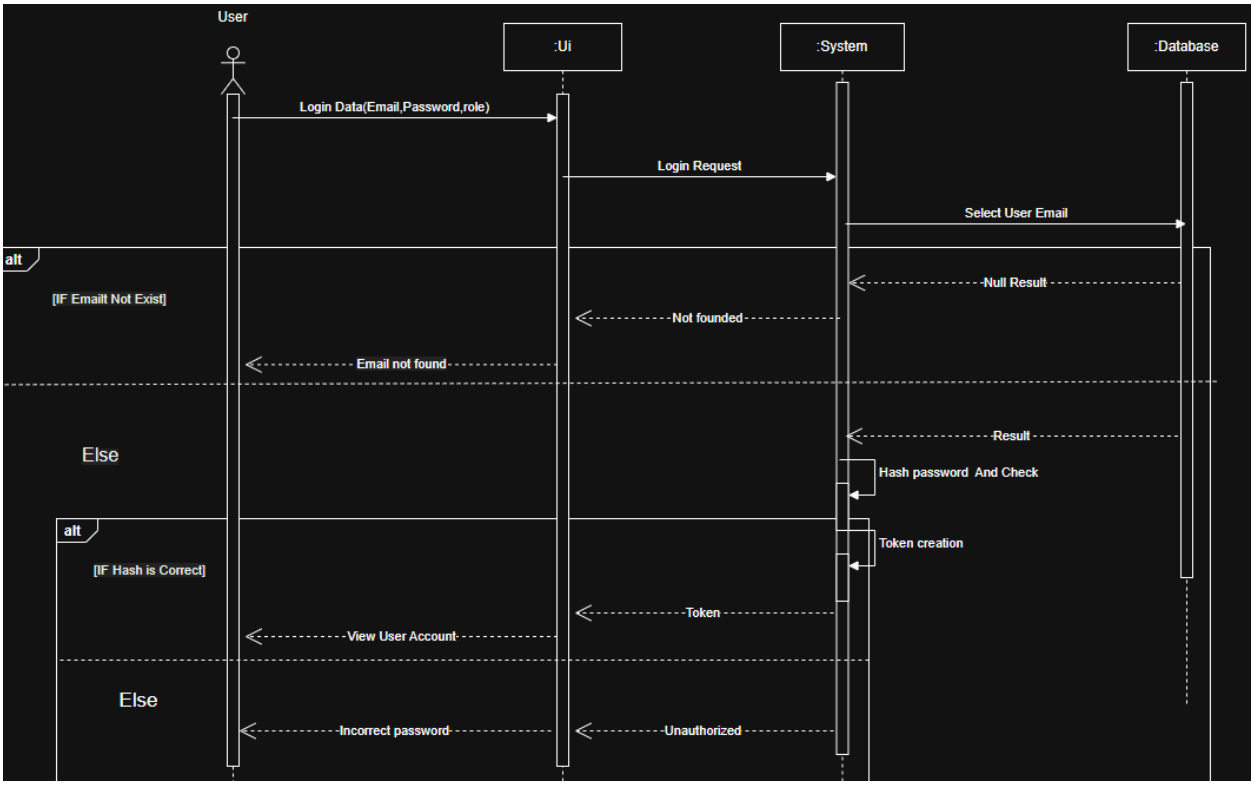
- **Payment Example:** User sends a payment request -> UI requests "Payment Service" -> Service communicates with the "Database" to update status -> Confirmation is sent back to the UI.
- **Booking Example:** Patient requests a booking -> "Appointment Controller" checks availability -> Database responds with status -> Booking is confirmed.

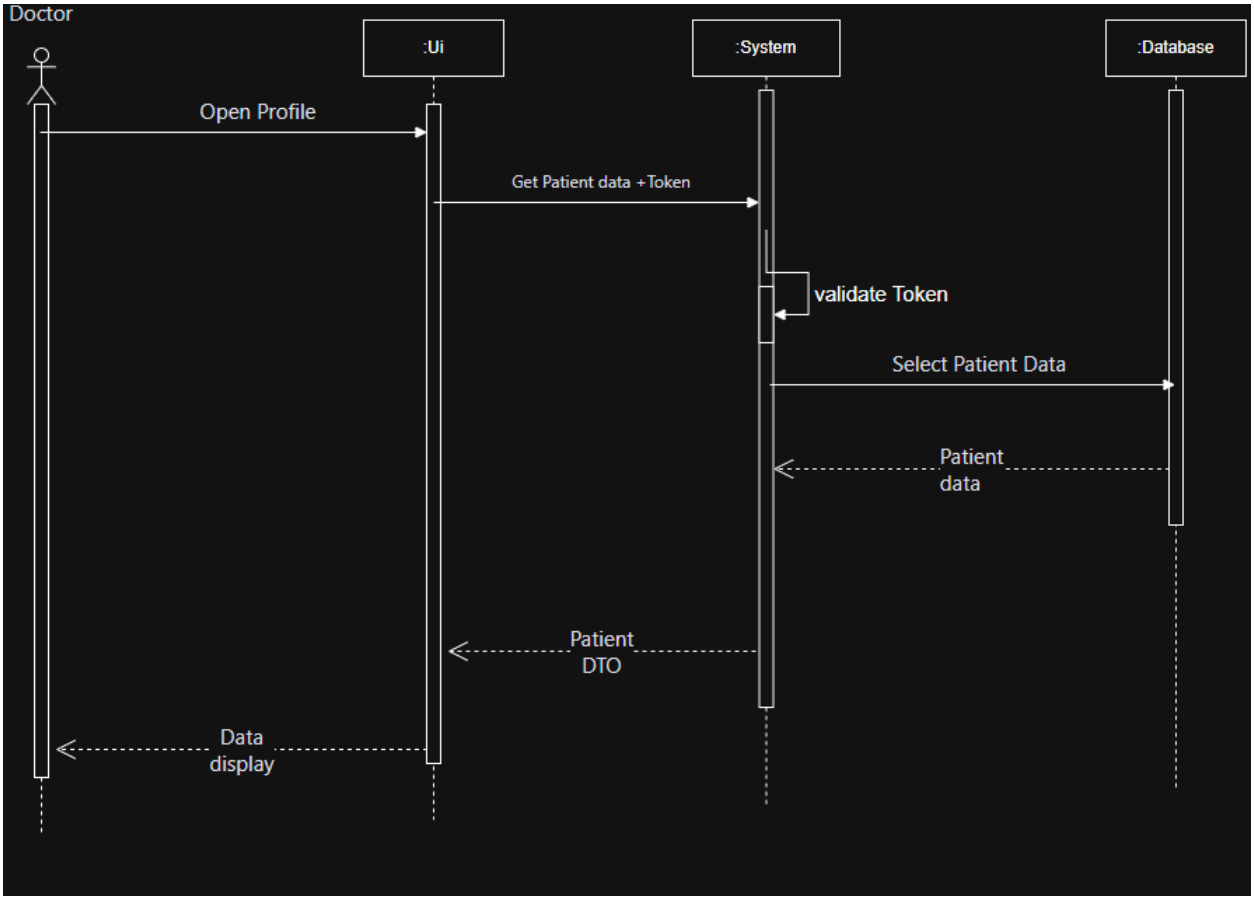
Sequence Diagram :

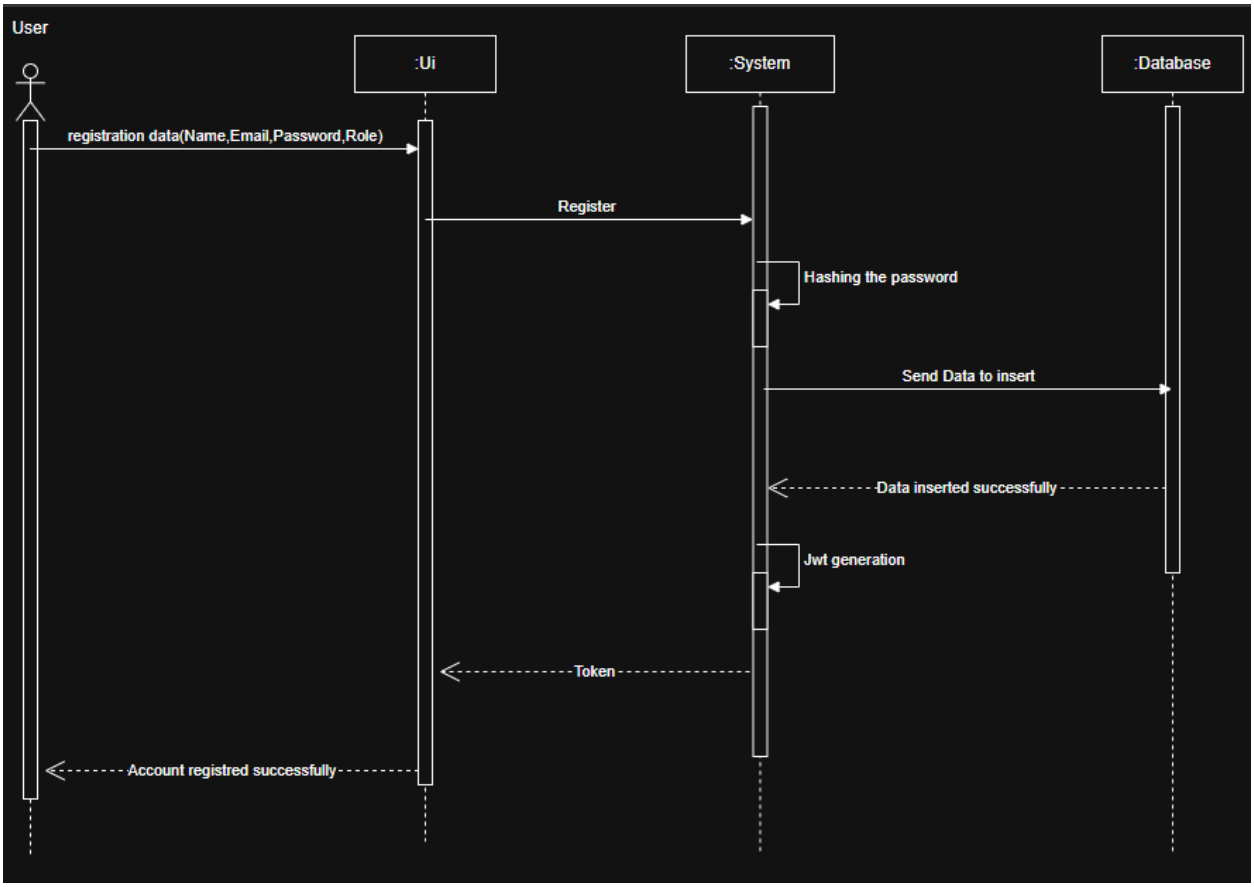

Patient
:Ui
:System
:Database
choose The doctor
Request The Time table of the doctor
Check Available Appointments
Return Available Appointments Table
Return Available Appointments Table
View Available Appointments and Ask for Select an Appointment
Choose the date
Request to book an appointment
Book an appointment and update its status
Booking success
Booking success
Booking success


User
:Ui
:Billing System
:Database
book an appointment
Request to create a bill
Insert a new bill
Addition success +Bill Number
Bill and price displaying
View Bill price
Choose a payment method and payment data
Confirm Payment
Bill status update
Updated
Payment receipt
View Receipt


Doctor
:Ui
:System
:Database
View My Schedule
Get Doctor Schedule + Token
validate Token
get Appointments Schedule+DoctorId
appointments schedule
appointments schedule
View
Schedule


User
:Ui
:System
:Database
Login Data(Email,Password,role)
Login Request
Select User Email
alt
[IF Emailt Not Exist]
Null Result
Not founded
Email not found
Else
Result
Hash password And Check
alt
[IF Hash is Correct]
Token creation
Token
View User Account
Else
Incorrect password
Unauthorized


Doctor
:Ui
:System
:Database
Open Profile
Get Patient data +Token
validate Token
Select Patient Data
Patient
data
Patient
DTO
Data
display



---

6. Non-Functional Requirements

- **Security:** Password encryption and the use of JWT (JSON Web Tokens).
- **Availability:** The system must maintain 99.9% uptime given its medical nature.
- **Usability:** Simple interfaces designed to accommodate elderly patients.

Functional Requirements

The system must satisfy the following functional requirements to ensure a seamless interaction between Patients, Doctors, and Administrators:

1. User Authentication & Authorization (Module 1)

- **FR-1.1:** The system shall allow users to register by choosing a role (Patient or Doctor) and providing valid credentials.
- **FR-1.2:** The system shall authenticate users via email and password, issuing a secure JWT token for session management.

- **FR-1.3:** The system shall provide a password recovery mechanism (Forgot Password) via email.
- **FR-1.4:** The Admin shall have the authority to manage (Create, Update, Delete) user accounts and roles.

2. Appointment Management (Module 3)

- **FR-2.1:** The system shall allow Patients to search for doctors based on name, specialty, or department.
- **FR-2.2:** The system shall display real-time availability of doctor time slots.
- **FR-2.3:** The system shall validate slot availability before confirming a booking to prevent double-booking.
- **FR-2.4:** The system shall allow users (Patients/Doctors) to reschedule or cancel appointments, automatically releasing the time slot.

3. Medical Profile & Directory (Modules 2 & 5)

- **FR-3.1:** Doctors shall be able to manage their profiles, including specialties, department, and contact info.
- **FR-3.2:** Patients shall be able to view their own profile and medical history.
- **FR-3.3:** The system shall maintain a searchable directory of all registered and verified doctors.

4. Billing & Financial Transactions (Module 4)

- **FR-4.1:** The system shall automatically generate an invoice after a medical consultation is completed.
- **FR-4.2:** The system shall calculate fees including base consultation rates and any additional charges.
- **FR-4.3:** The system shall support multiple payment methods and update the payment status to "Paid" upon success.
- **FR-4.4:** The system shall generate and display a digital receipt for every transaction.

5. System Auditing & Logging (Module 6 - AOP)

- **FR-5.1:** The system shall use Aspect-Oriented Programming (AOP) to log all user activities and API requests.

- **FR-5.2:** The system shall record and log the execution time of each operation for performance monitoring.
- **FR-5.3:** The Admin shall be able to view audit logs to monitor system security and performance.

---

8. Conclusion

Based on the provided models, the system demonstrates strong integration between administrative and technical processes. The use of AOP ensures a high level of oversight, while the sequence and activity diagrams provide a clear roadmap for developers to build the system without logical errors.